David Mohaisen, Professor
University of Central Florida
Department of Computer Science
Cybersecurity and Privacy Cluster

https://cs.ucf.edu/~mohaisen/

Probabilistic Language Modeling in NLP

① Language Modeling Basics
② N-Gram Models
③ Sparsity Challenges
④ Smoothing Techniques
⑤ Neural Language Models
⑥ Fixed-Window Models

Objectives

- Introduction to language models and their applications
- The basic notion of language models.
- N-grams as language models
- Problems with n-grams and their solutions
- Introduction to window-based and neural language models

Covered Concepts…

- **Word2Vec:** NN-based method for learning word embeddings that capture semantics. Produces dense vector representations (CBOW & Skip-gram).
- **One-Hot Representation:** A sparse vector where one dimension is (1) while others are 0. Useful for simple encoding but lacks semantic relationships.
- **Named Entity Resolution (NER):** Identifying and classifying proper names (people, places, organizations) in text for information extraction.
- **Window Classification:** A method for NLP tasks where a fixed-size context window around a word is used for classification (e.g., POS tagging, NER).
- **Artificial Neural Networks (ANN):** Deep learning models used in NLP for tasks like sentiment analysis, text generation, and machine translation.
- **Language Dependency & Grammar:** Challenges arise due to language-specific syntactic structures and grammar rules, requiring advanced models for cross-linguistic generalization.

Today: Language Model

- **What is Language Modeling?**
  - Language modeling is the task of predicting the next word in a sequence based on previous words. Formally, given a sequence of words $\{w_1, w_2, \ldots, w_{n-1}\}$, a language model computes the probability of the next word w_n.

Why Language Models

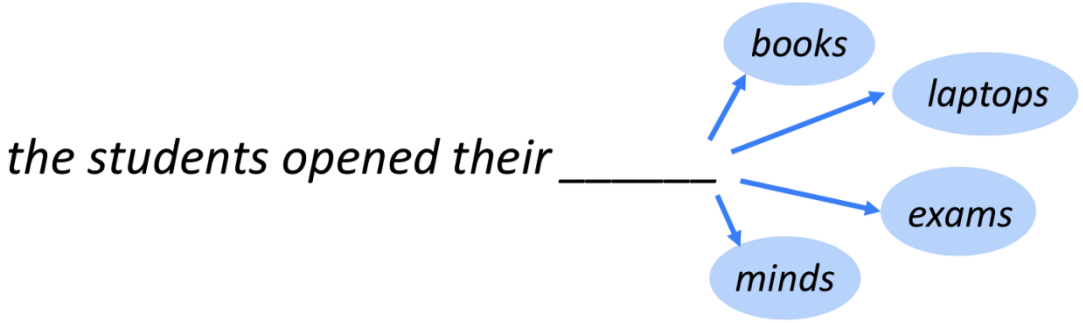
- **Understanding Context:** Helps machines grasp semantic meaning and syntactic structure in natural language.
- **Foundation of NLP:** Many NLP tasks (e.g., speech recognition, translation) depend on accurate language modeling.
- **Improving Human-Computer Interaction:** Enables more natural conversations in AI-driven chatbots and virtual assistants.
- **Reducing Ambiguity:** Enhances disambiguation in text processing, making responses more accurate.
- **Enhancing Search & Information Retrieval:** Helps improve the relevance of search results based on user queries.

Applications of LM

- **Speech Recognition:** Predicts the most likely words based on spoken input (e.g., Siri, Google Assistant).
- **Machine Translation:** Improves fluency and accuracy of translations by predicting contextually appropriate words.
- **Text Autocompletion & Prediction:** Powers predictive text in search engines, messaging apps, and word processors.
- **Chatbots & Virtual Assistants:** Enhances conversational AI by predicting contextually relevant responses.
- **Summarization & Text Generation:** Used in models like GPT to generate coherent and meaningful text.
- **Optical Character Recognition (OCR):** Helps correct errors by predicting missing or unclear words.

Language modeling

- Language Modeling is the task of predicting what word comes next



- More formally: given a sequence of words $x^{(1)}, x^{(2)}, x^{(3)}, \ldots x^{(t)}$, compute the probability distribution of the next word $x^{(t+1)}$
  - Where $x^{(t+1)}$ can be any word in the vocabulary $V = \{w_1, \ldots, w_{|v|}\}$.
  - A system that does that is called a language model (LM).

Language modeling

- Think of a language model as a system that assigns probability to a piece of text
  - For example, if we have some text $x^{(1)}, x^{(2)}, x^{(3)}, \ldots x^{(T)}$, then the probability of this text (according to the language model is:

$$P(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}) = P(\boldsymbol{x}^{(1)}) \times P(\boldsymbol{x}^{(2)} | \boldsymbol{x}^{(1)}) \times \cdots \times P(\boldsymbol{x}^{(T)} | \boldsymbol{x}^{(T-1)}, \ldots, \boldsymbol{x}^{(1)})$$

$$= \prod_{t=1}^{T} \underbrace{P(\boldsymbol{x}^{(t)} | \boldsymbol{x}^{(t-1)}, \ldots, \boldsymbol{x}^{(1)})}_{\text{This is what our LM provides}}$$

How can we do that?

- **The student opened their _____**
  - Possible completions: *book, laptop, notes, email, eyes, browser, assignment*
  - The correct choice depends on **context** and **probability of word sequences** in natural language.

The n-gram Language Model

- **Answer**: use n-gram language models (pre-deep learning approach).
  - **Definition**: a n-gram is a chunk of n consecutive words
    - unigrams: “the”, “students”, “opened”, ”their”
    - bigram: ”the students”, “students opened”, ”opened their”
    - trigram: “the students opened” ”students opened their”
    - 4-gram: “the students opened their”

The n-gram Language Model, cont.

- How to predict the next word?
  - Collect statistics about how frequent different n-grams are, and use these statistics to predict the next word.

The n-gram Language Model, cont.

- First, we make a simplifying assumption: $x^{(t+1)}$ depends only on the preceding n-1 words.

$$P(\boldsymbol{x}^{(t+1)}|\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(1)}) = P(\boldsymbol{x}^{(t+1)}|\overbrace{\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)}}^{n\text{-1 words}}) \qquad \text{(assumption)}$$

prob of a n-gram

$$= \frac{\boxed{P(\boldsymbol{x}^{(t+1)},\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)})}}{\boxed{P(\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)})}} \qquad \text{(definition of conditional prob)}$$

prob of a (n-1)-gram

- Question: how to get the n-gram and (n-1)-gram probability
- Answer: by counting them in some large text corpus.

The n-gram Language Model, cont.

- Suppose we are learning a 4-gram language model

~~as the proctor started the clock, the~~ students opened their _____

discard

condition on this

$$P(\boldsymbol{w}|\text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

- For example, suppose that the corpus:
  - "students opened their" occurred 1000 times
  - "students opened their books" occurred 400 times
    - P(books | students opened their) = 0.4
  - "students opened their exams" occurred 100 times
    - P(exams | students opened their) = 0.1

Problem 1: Sparsity in n-grams

- Definition: Many valid N-Grams never appear in training data due to limited samples. Example:
  - Training data: *"The cat sat on the mat."*
  - Missing bigram: *"cat jumped"* → No probability estimate.
- **Cause**: As ***n*** increases, the number of possible N-Grams grows **exponentially**, leading to many unseen combinations.
- **Why Does Sparsity Occur?**

  1. **Finite Training Data** – Large vocabulary but limited examples per N-Gram.
  2. **Rare/New Phrases** – Some valid word sequences never appear in training.
  3. **Exponential Growth** – A vocabulary of 10,000 words leads to:
     1. **Bigram space**: 10^8 combinations
     2. **Trigram space**: 10^{12} combinations

Problem 1: Sparsity in n-grams, cont.

Sparsity Problem 1

Problem: what if "students opened their w" never occurred in data? Then w has a probability of 0

(Partial) Solution: Add small delta to the count of every w in V; i.e., smoothing.

$$P(w \mid \text{students opened their}) = \frac{\text{Count(students opened their } w)}{\text{Count(students opened their)}}$$

Sparsity Problem 2

Problem: what if "students opened their" never occurred in data? Then we can't calculate probability for any w

(Partial) Solution: just condition on "opened their" instead; i.e., backoff.

Note: increasing n makes sparsity problems worse. Typically we cannot have n larger than 5.

General Solutions to Sparsity

- **Smoothing Techniques**
  - **Laplace Smoothing** – Adds a small constant to all word counts to avoid zero probabilities.
  - **Kneser-Ney Smoothing** – Adjusts probabilities based on how often words appear in different contexts.
- **Backoff & Interpolation**
  - Use lower-order *n*-grams when higher-order ones are missing.
  - Example: If a three-word sequence (trigram) is not found, use the probability of a two-word sequence (bigram) instead.
- **Word Embeddings (e.g., Word2Vec, BERT)**
  - Represent words as continuous numerical vectors that capture meaning and relationships.
  - Help models understand words based on similarity rather than relying on exact matches.
- **Neural Language Models (LSTMs, Transformers)**
  - Learn word relationships using deep learning instead of just counting N-Grams.
  - Improve performance by generalizing better to new or unseen word sequences.

Problem 2: Storage in n-grams

<u>Storage</u>: need to store count for all n-grams you saw in the corpus

$$P(w \mid \text{students opened their}) = \frac{\text{Count(students opened their } w\text{)}}{\text{Count(students opened their)}}$$

Increasing n or increasing the corpus size increases the model size.

The n-grams in Practice

- You can build a simple trigram Language Model over a 1.7 million word corpus (Reuters) in a few seconds on your laptop.
  - Today the _____

get probability distribution

company	0.153
bank	0.153
price	0.077
italian	0.039
emirate	0.039

Sparsity problem: not much granularity in the probability distribution

The n-grams in Practice, cont.

- You can build a simple trigram Language Model over a 1.7 million word corpus (Reuters) in a few seconds on your laptop.
  - Today the _____

Today the price _____

Conditioned on

get probability distribution

company		0.153
bank		0.153
price	**Sample**	0.077
italian		0.039
emirate		0.039

Conditioned on

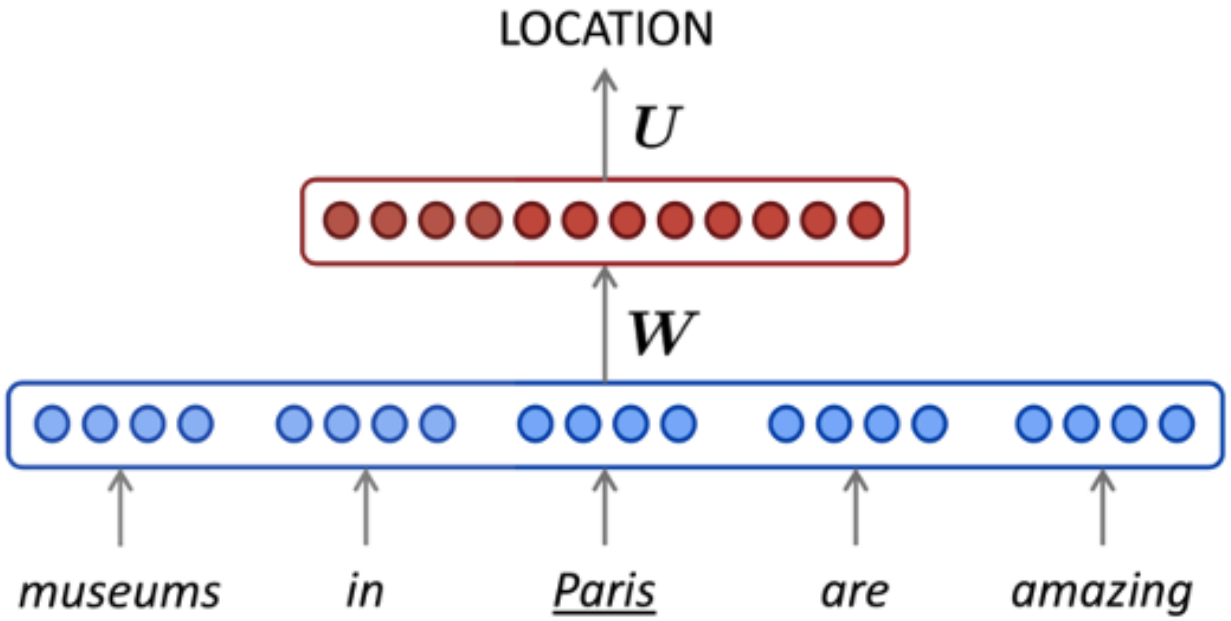
of	**Sample**	0.381
for		0.050
it		0.041
to		0.039
is		0.031

Generating text with n-gram

- *today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .*
  - *Grammar is okay, but incoherent.*

Neural Language Models

- Recall the language model task:
  - Input: sequence of words $x^{(1)}, x^{(2)}, \ldots x^{(t)}$
  - Output: prob. dist. of the next word $P(x^{(3)} \mid x^{(1)}, x^{(2)}, \ldots x^{(t)})$
- How about the window-based neural model?
  - We saw this applied to named entity recognition (NER)



Fixed-Window Language Model

~~as the proctor started the clock~~ the students opened their _____

discard

fixed window

Fixed-Window Language Model

output distribution

$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{U}\boldsymbol{h} + \boldsymbol{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

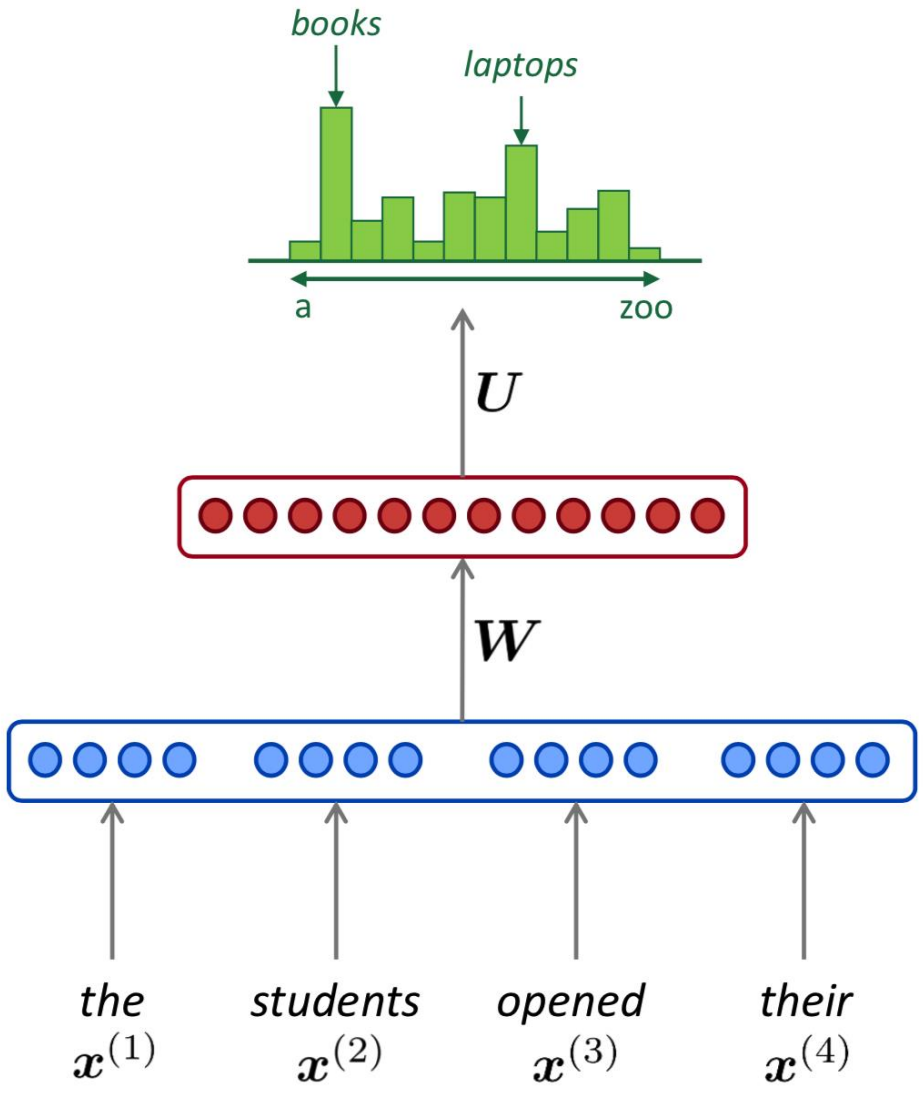
$$\boldsymbol{h} = f(\boldsymbol{W}\boldsymbol{e} + \boldsymbol{b}_1)$$

concatenated word embeddings

$$\boldsymbol{e} = [\boldsymbol{e}^{(1)}; \boldsymbol{e}^{(2)}; \boldsymbol{e}^{(3)}; \boldsymbol{e}^{(4)}]$$

words / one-hot vectors

$$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$$

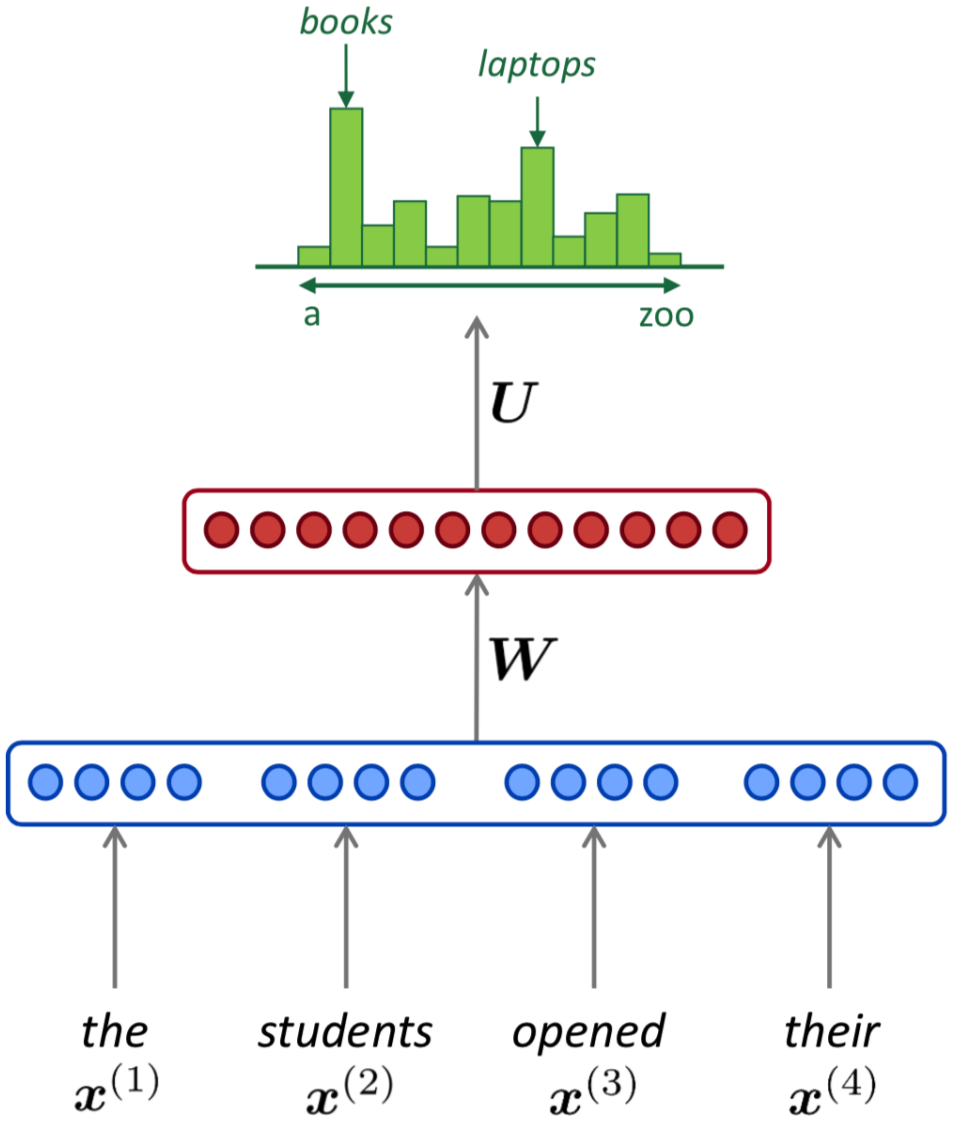


Fixed-Window Language Model

Improvements over *n*-gram LM:

- No sparsity problem
- Don't need to store all observed *n*-grams

Remaining **problems**:

- Fixed window is too small
- Enlarging window enlarges $\boldsymbol{W}$
- Window can never be large enough!
- $\boldsymbol{x}^{(1)}$ and $\boldsymbol{x}^{(2)}$ are multiplied by completely different weights in $\boldsymbol{W}$. No symmetry in how the inputs are processed.

We need a neural architecture that can process *any length input*